STAR SCHEMA

A Complete Data Warehousing Guide with SQL, PySpark & Databricks Examples

What you will learn

- Star Schema fundamentals and architecture
- Fact tables, dimension tables, and fact-table grain
- Business keys, surrogate keys, and SCD Type 2
- SQL and PySpark implementation
- Star Schema vs. Snowflake Schema
- How dimensional modelling fits into Medallion Architecture
- Important interview concepts and questions

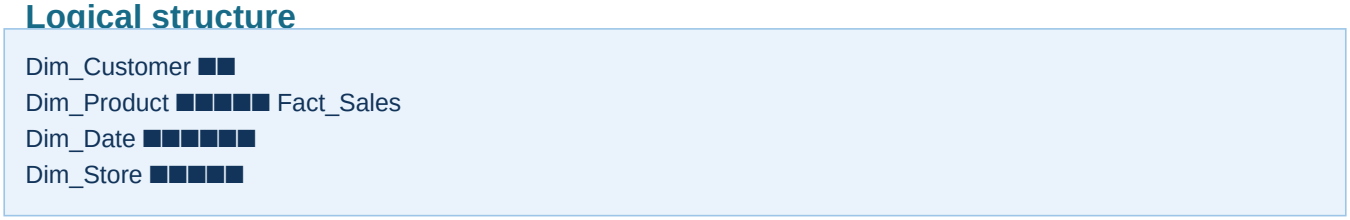
Core idea: A fact table stores measurable business events; dimension tables provide the descriptive context used to analyze those events.

1. What is a Star Schema?

A **Star Schema** is a dimensional modelling technique used in data warehouses. It places a central fact table at the centre and connects it directly to multiple dimension tables. The layout resembles a star.

For an e-commerce business, a typical model contains Fact_Sales connected to Dim_Customer, Dim_Product, Dim_Date, and Dim_Store.

Logical structure



The fact table stores measures such as quantity and sales amount, plus keys that connect to dimensions. Dimension tables store descriptive attributes such as customer name, product category, city, and calendar month.

Why use it?

• Makes business questions easier to express with SQL.

• Supports filtering, grouping, slicing, and aggregation.

• Often reduces the number of joins compared with highly normalized dimensional models.

• Provides a consistent structure for BI tools and analytical workloads.

2. Fact Tables

A **fact table** stores business events or measurements. It usually contains foreign keys to dimensions and numeric measures that can be analyzed.

sales_id	customer_sk	product_sk	date_sk	quantity	sales_amount
1	101	501	20261001	2	2000
2	102	502	20261001	1	1500
3	101	503	20261002	3	3000

The fact table's foreign keys connect each sale to the relevant customer, product, and date dimension records. Measures such as quantity and sales_amount support aggregation.

Three common fact-table types

Type	What one row represents	Example
Transaction fact	An individual business event or transaction	One order line or bank transaction
Periodic snapshot	The state of something at a regular interval	Daily account balance or monthly inventory
Accumulating snapshot	Progress through a process with milestones	Order placed, shipped, and delivered dates

Transaction fact: Usually records each event at its lowest useful detail. **Periodic snapshot:** records a measurement at fixed intervals. **Accumulating snapshot:** is updated as a process moves through known milestones.

3. Dimension Tables

A **dimension table** contains descriptive attributes that provide context for facts. It answers questions such as who, what, when, where, and how.

Example: Dim_Customer

customer_sk	customer_id	customer_name	city	state
1	C101	Ravi	Hyderabad	Telangana
2	C102	Kumar	Chennai	Tamil Nadu
3	C103	Raj	Bengaluru	Karnataka

Example: Dim_Product

product_sk	product_id	product_name	category	brand
501	P501	Laptop	Electronics	Dell
502	P502	Mobile	Electronics	Samsung
503	P503	Chair	Furniture	Nilkamal

Fact table vs. dimension table

Feature	Fact table	Dimension table
Purpose	Stores measurable events	Stores descriptive context
Typical contents	Dimension keys and measures	Names, categories, dates, locations
Rows	Often very large	Usually smaller
Example	Fact_Sales	Dim_Customer
Common use	SUM, COUNT, AVG and other analysis	Filtering, grouping, and labeling

4. Grain: Define It Before Building the Fact

Grain states exactly what one row in a fact table represents. It is one of the most important design decisions in dimensional modelling.

Order-level grain

order_id	total_quantity	total_amount
1001	6	6000

One row represents an entire order. This is useful for order-level analysis, but it does not retain the detail of each product line.

Order-item-level grain

order_id	product_id	quantity	amount
1001	501	2	2000
1001	502	1	1500

order_id	product_id	quantity	amount
1001	503	3	2500

One row represents one product line within an order. This supports product-level analysis.

Design rule: State the grain in a sentence before choosing columns. An unclear grain can cause duplicate counting, incorrect joins, and inaccurate aggregations.

5. Keys: Business, Surrogate, Primary, and Foreign

Business key / natural key

A business key comes from the source system and identifies a business entity, for example customer_id = C101. It may be meaningful to the business and can remain the same when descriptive attributes change.

Surrogate key

A surrogate key is an artificial warehouse-generated identifier. In a Type 2 dimension, each historical version of the same business entity receives a different surrogate key.

customer_sk	customer_id	name	city	is_current
1	C101	Ravi	Hyderabad	false
2	C101	Ravi	Mumbai	true

The business key C101 remains the same. The surrogate key changes from 1 to 2 because these rows represent different historical versions.

Primary key and foreign key

A primary key uniquely identifies a row in a table. A foreign key in the fact table points to a dimension row, commonly through its surrogate key.

Why surrogate keys matter: They allow the fact table to reference the exact dimension version that was valid when the business event occurred.

6. Star Schema and SCD Type 2

Slowly Changing Dimension (SCD) techniques define how dimension attribute changes are stored. Type 2 preserves history by expiring the old version and inserting a new version.

customer_sk	customer_id	city	start_date	end_date	current
1	C101	Hyderabad	2026-01-01	2026-06-30	No
2	C101	Mumbai	2026-07-01	NULL	Yes

Suppose Ravi lived in Hyderabad when sale 1001 occurred and moved to Mumbai before sale 1002.

sales_id	customer_sk	sales_amount
1001	1	50000
1002	2	30000

Sale 1001 references the Hyderabad version; sale 1002 references the Mumbai version. This allows historical reports to reflect the customer's attributes at the time of each sale.

Important: The fact table should resolve the correct dimension surrogate key using the event date and the dimension's effective date range. If it always looks up the current customer row, historical reporting can become incorrect.

7. Implementing a Star Schema with SQL

The following simplified example uses SQL types supported by common Spark SQL / Databricks environments. Production tables should also define appropriate constraints, storage properties, and key-generation logic.

Create dimension tables

```
CREATE TABLE dim_customer (
    customer_sk BIGINT,
    customer_id STRING,
    customer_name STRING,
    city STRING,
    state STRING
) USING DELTA;

CREATE TABLE dim_product (
    product_sk BIGINT,
    product_id STRING,
    product_name STRING,
    category STRING
) USING DELTA;

CREATE TABLE dim_date (
    date_sk INT,
    full_date DATE,
    month STRING,
    year INT
) USING DELTA;
```

Create the fact table

```
CREATE TABLE fact_sales (
    sales_id BIGINT,
    customer_sk BIGINT,
    product_sk BIGINT,
    date_sk INT,
    quantity INT,
    sales_amount DOUBLE
) USING DELTA;
```

Insert sample data

```
INSERT INTO dim_customer VALUES
(1, 'C101', 'Ravi', 'Hyderabad', 'Telangana'),
(2, 'C102', 'Kumar', 'Chennai', 'Tamil Nadu');

INSERT INTO dim_product VALUES
(501, 'P501', 'Laptop', 'Electronics'),
(502, 'P502', 'Mobile', 'Electronics');

INSERT INTO dim_date VALUES
(20261001, DATE '2026-10-01', 'October', 2026);

INSERT INTO fact_sales VALUES
(1001, 1, 501, 20261001, 2, 100000),
(1002, 2, 502, 20261001, 1, 30000);
```

Query: sales by customer and product category

```
SELECT
    c.customer_name,
    p.category,
    SUM(f.sales_amount) AS total_sales
FROM fact_sales f
JOIN dim_customer c
    ON f.customer_sk = c.customer_sk
JOIN dim_product p
    ON f.product_sk = p.product_sk
GROUP BY
    c.customer_name,
    p.category;
```

customer_name	category	total_sales
Ravi	Electronics	100000
Kumar	Electronics	30000

8. Implementing the Model with PySpark

In PySpark, each dimension and fact table can be represented as a DataFrame. The DataFrames can be written to Delta tables and joined for analysis.

Create a customer dimension DataFrame

```
customer_data = [
    (1, "C101", "Ravi", "Hyderabad", "Telangana"),
    (2, "C102", "Kumar", "Chennai", "Tamil Nadu")
]

customer_columns = [
    "customer_sk", "customer_id", "customer_name",
    "city", "state"
]

dim_customer = spark.createDataFrame(
    customer_data, customer_columns
)
```

Create a product dimension DataFrame

```
product_data = [
    (501, "P501", "Laptop", "Electronics"),
    (502, "P502", "Mobile", "Electronics")
]

product_columns = [
    "product_sk", "product_id",
    "product_name", "category"
]

dim_product = spark.createDataFrame(
    product_data, product_columns
)
```

Create a fact DataFrame

```
sales_data = [
    (1001, 1, 501, 20261001, 2, 100000.0),
    (1002, 2, 502, 20261001, 1, 30000.0)
]

sales_columns = [
    "sales_id", "customer_sk", "product_sk",
    "date_sk", "quantity", "sales_amount"
]

fact_sales = spark.createDataFrame(
    sales_data, sales_columns
)
```

Join and aggregate

```
from pyspark.sql.functions import sum as sum_

result_df = (
    fact_sales.alias("f")
    .join(dim_customer.alias("c"), "customer_sk", "inner")
    .join(dim_product.alias("p"), "product_sk", "inner")
)

final_df = (
    result_df
    .groupBy("customer_name", "category")
    .agg(sum_("sales_amount").alias("total_sales"))
)
```

```
display(final_df)
```

For production workloads, define explicit schemas, generate stable surrogate keys, validate uniqueness, handle unmatched dimension keys, and write the DataFrames to managed or external Delta tables as appropriate.

9. Star Schema vs. Snowflake Schema

A Star Schema connects dimensions directly to the fact table. A Snowflake Schema further normalizes dimension data into related sub-dimension tables.

Feature	Star Schema	Snowflake Schema
Structure	Fact connected directly to dimensions	Dimensions split into related tables
Normalization	Usually denormalized dimensions	More normalized
Joins	Generally fewer	Generally more
Query complexity	Often simpler for BI	Can be more complex
Redundancy	May repeat descriptive values	Can reduce repetition
Maintenance	Often simpler to query	More relationships to maintain

Example: In a Snowflake model, customer location may be separated into Dim_City and Dim_State. In a Star Schema, city and state may simply be attributes in Dim_Customer.

Choice depends on the workload: Star Schema is commonly used for straightforward analytical and BI models. Snowflake Schema can be useful where normalization and shared hierarchies are important.

10. Star Schema in Medallion Architecture

Medallion Architecture describes how data is processed through Bronze, Silver, and Gold layers. Star Schema describes how data is organized for analytics. They solve different problems and work together.

Layer	Typical responsibility	Example
Bronze	Land raw source data with minimal changes	Raw customers, products, orders
Silver	Clean, validate, deduplicate, standardize	Clean customer, product, order datasets
Gold	Publish business-ready analytical models	Fact_Sales, Dim_Customer, Dim_Product, Dim_Date

A common architecture is to ingest source data into Bronze, transform and validate it in Silver, then create Gold fact and dimension tables for BI and analytics.

11. Additional Dimensional Modelling Concepts

Additive, semi-additive, and non-additive measures

Measure type	Meaning	Example
Additive	Can be summed across all relevant dimensions	Sales amount, units sold
Semi-additive	Can be summed across some dimensions but not time	Account balance, inventory level

Measure type	Meaning	Example
Non-additive	Cannot be meaningfully summed directly	Percentage, ratio, margin

For ratios such as profit margin, calculate from underlying additive values (for example, total profit divided by total revenue) rather than summing row-level percentages.

Conformed dimensions

A conformed dimension is shared consistently by multiple fact tables. For example, Fact_Sales and Fact_Returns can both use the same Dim_Date and Dim_Product.

Role-playing dimensions

One dimension is used in multiple roles. A single Dim_Date can represent order date, shipment date, and delivery date through separate foreign-key relationships.

Degenerate dimensions

A business identifier, such as order_number, is stored directly in a fact table without a separate dimension table when it has no additional descriptive attributes requiring a dimension.

Factless fact tables

A factless fact table records that an event or relationship occurred, even when there are no numeric measures. Examples include student attendance and promotional eligibility.

12. Best Practices

• **Define the grain first.** Write down exactly what one fact row represents.

• **Choose keys deliberately.** Use business keys to identify source entities and surrogate keys to identify warehouse dimension versions.

• **Handle unknown or missing dimensions.** Consider an explicit Unknown member so fact rows are not silently lost when a dimension lookup fails.

• **Preserve historical accuracy.** For Type 2 dimensions, resolve the correct surrogate key using the event date and effective date range.

• **Use consistent dimensions.** Reuse conformed dimensions across fact tables when business definitions match.

• **Validate joins and measures.** Check row counts, uniqueness, referential coverage, and aggregation totals.

• **Do not sum every measure.** Understand whether a measure is additive, semi-additive, or non-additive.

• **Design for query patterns.** Include useful descriptive attributes and avoid unnecessary joins for common reporting needs.

13. Interview Questions

Q. What is a Star Schema?

A. A dimensional model with a central fact table connected directly to descriptive dimension tables.

Q. What is the difference between a fact and a dimension?

A. Facts store measurable events; dimensions provide descriptive context.

Q. What is grain?

A. The precise business meaning of one row in a fact table.

Q. Why define grain first?

A. It prevents inconsistent detail, duplicate counting, and incorrect aggregation.

Q. What are the three common fact-table types?

A. Transaction, periodic snapshot, and accumulating snapshot.

Q. Why use surrogate keys?

A. They uniquely identify dimension versions, including Type 2 historical versions.

Q. How does SCD Type 2 work with a Star Schema?

A. It creates a new dimension version and facts reference the version valid at the event time.

Q. What is a conformed dimension?

A. A shared dimension used consistently across multiple fact tables.

Q. What is a role-playing dimension?

A. A dimension reused for multiple roles, such as order date and delivery date.

Q. What is a degenerate dimension?

A. A business identifier stored in the fact table without a separate dimension table.

Q. What is the difference between Star and Snowflake?

A. Star dimensions are generally denormalized; Snowflake dimensions are further normalized.

Q. How does Star Schema fit into Medallion Architecture?

A. It is commonly published in Gold; Medallion describes processing layers, while Star Schema describes analytical modelling.

14. Quick Revision

Concept	Remember
Star Schema	Central fact table connected to dimension tables
Fact table	Business events, measures, and dimension keys
Dimension table	Descriptive attributes for filtering and grouping
Grain	What one fact row represents
Business key	Identifier from the source/business domain
Surrogate key	Warehouse-generated identifier for a dimension row/version
SCD Type 1	Overwrite; no previous value retained in that dimension row
SCD Type 2	Insert a new version and preserve historical versions
Conformed dimension	Shared dimension used by multiple facts
Gold layer	Common place for business-ready fact and dimension models

Final takeaway: Star Schema is a way to structure analytical data. Start with the business process and grain, identify facts and dimensions, choose appropriate keys, define history requirements, and then implement and validate the model.